

Tabela com as respostas

Teste	Método	URL	Payload	Status	Esperado
#1	GET	/usuarios	nenhum	200	sim
#2	POST	/usuarios	clique aqui	201	sim
#3	POST	/usuarios	clique aqui	400	sim (no caso o 400)
#4	POST	/usuarios	clique aqui	500	sim
#5	PUT	/usuarios/2	clique aqui	200	sim
#6	PUT	/usuarios/9999	indiferente	404	sim
#7	DELETE	/usuarios/8	indiferente	200	sim
#8	DELETE	/usuarios/9999	indiferente	404	sim

Sumário

Tabela com as respostas.....	1
Teste #1 - Listagem de usuários.....	2
Teste #2 - Criação de usuário válido.....	3
Teste #3 - Criação de usuário inválido.....	4
Teste #4 - Criação de usuário com email duplicado.....	5
Teste #5 - Atualização de usuário.....	6
Teste #6 - Atualização de usuário inexistente.....	7
Teste #7 - Remover um usuário existente.....	8
Teste #8 - Remover um usuário inexistente.....	8

Em cada teste há prints e uma breve explicação do que foi feito.

Teste #1 - Listagem de usuários

- **Método:** GET
- **Endpoint:** /usuarios
- **Payload:** nenhum

Ao fazermos essa requisição, temos o código de status 200, já que houve sucesso numa requisição que não envolve a criação de nenhum tipo de dado.

O resultado é o seguinte:

```
{
  "id": 1,
  "nome": "Nome 1",
  "email": "nome1@dominio.com"
},
{
  "id": 2,
  "nome": "Nome 2",
  "email": "nome2@dominio.com"
},
{
  "id": 3,
  "nome": "Nome 3",
  "email": "nome3@dominio.com"
},
{
  "id": 6,
  "nome": "Nome 4",
  "email": "nome4@dominio.com"
}
```

(o último usuário tem ID 6, pois eu havia testado antes e deletado alguns usuários...)

Teste #2 - Criação de usuário válido

- Método: POST
- Endpoint: /usuarios
- Payload: objeto com campos nome e email

Ao fazermos essa requisição, enviando um corpo JSON com dados corretos, temos uma resposta com código de status 201, pois tivemos sucesso em uma operação que insere dados.

Ao enviarmos o seguinte payload:

```
{
  "nome": "Nome 5",
  "email": "nome5@dominio.com"
}
```

Temos o seguinte resultado:

Status: 201 Created Size: 52 Bytes Time: 33 ms

Response Headers 7 Cookies Results Docs

```
1 {
2   "id": 8,
3   "nome": "Nome 5",
4   "email": "nome5@dominio.com"
5 }
```

Teste #3 - Criação de usuário inválido

- Método: POST
- Endpoint: /usuarios
- Payload: objeto com campo nome

Ao fazermos essa requisição, enviando um corpo JSON com dados faltando, temos uma resposta com código de status 400, pois fizemos uma má requisição. Sobre a pergunta, o ideal seria o 400 mesmo, pois ele se refere a uma má requisição, que é exatamente o caso desse teste. O erro não é do servidor e sim do cliente, que mandou dados errados.

Ao enviarmos o seguinte payload:

```
{  
  "nome": "Nome 6"  
}
```

Temos o seguinte resultado:

Status: 400 Bad Request Size: 51 Bytes Time: 14 ms

Response	Headers 7	Cookies	Results	Docs
1 {				
2 "message": "Missing required fields (name/email)."				
3 }				

Teste #4 - Criação de usuário com email duplicado

- Método: POST
- Endpoint: /usuarios
- Payload: objeto com campo email duplicado

Ao fazermos essa requisição, enviando um corpo JSON com o campo email já existente no banco, temos um código de status 500, já que dá erro de UNIQUE no banco de dados. Respondendo a pergunta, o código de status ideal para esse caso seria algo como 409, já que se trata de um erro do cliente enviando um dado já duplicado (não respeitando as restrições).

Ao enviarmos o seguinte payload:

```
{
  "nome": "Nome 5",
  "email": "nome5@dominio.com"
}
```

Temos o seguinte resultado:

Status: 500 Internal Server Error Size: 72 Bytes Time: 28 ms

Response

Headers 7

Cookies

Results

Docs

</> ≡

```
1 {
2   "error": "Duplicate entry 'nome5@dominio.com' for key 'usuarios.email'"
3 }
```

Teste #5 - Atualização de usuário

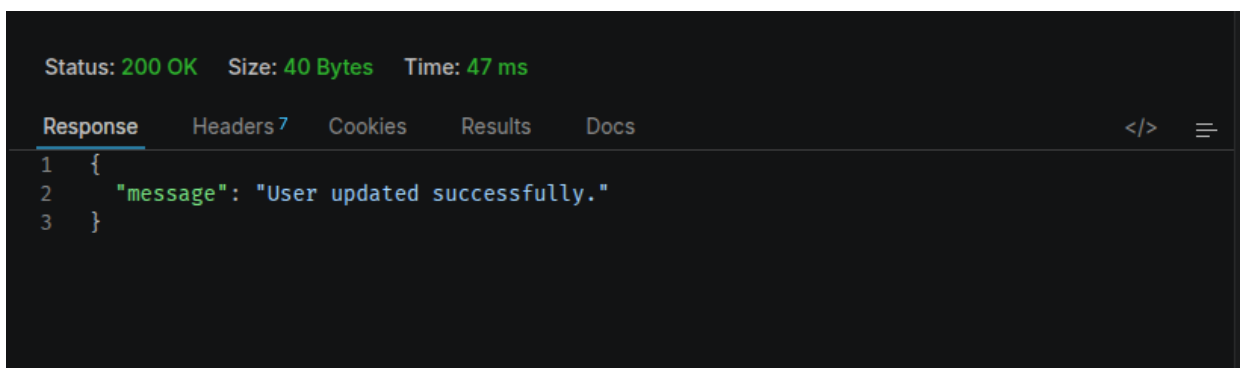
- Método: PUT
- Endpoint: /usuarios/2
- Payload: nome novo

Ao fazermos essa requisição, temos código de status 200, já que tivemos sucesso na atualização (não criação) de um usuário.

Ao enviarmos o seguinte payload:

```
{
  "nome": "Ana Souza Silva"
}
```

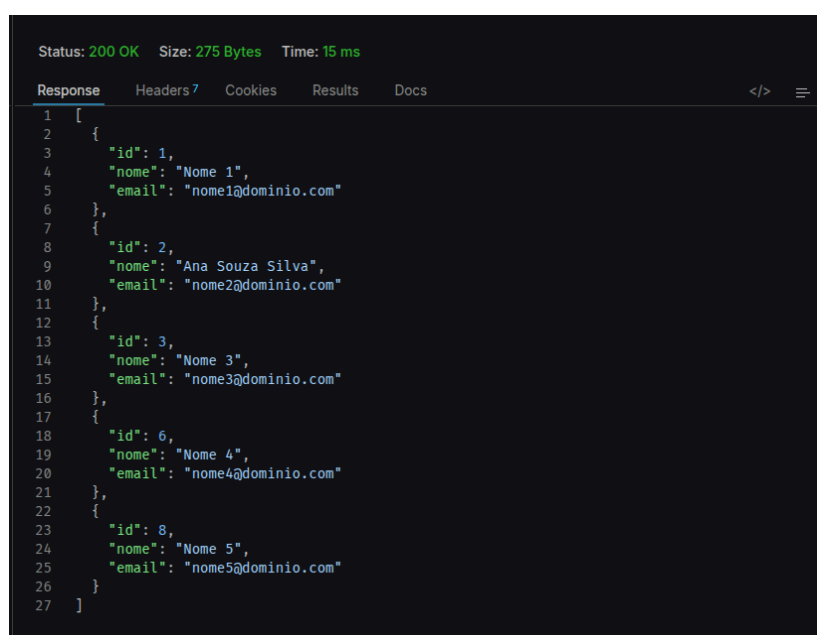
Temos o seguinte resultado:



The screenshot shows a REST client interface with the following details:

- Status: 200 OK
- Size: 40 Bytes
- Time: 47 ms
- Response tab is selected, showing the following JSON:

```
1 {
2   "message": "User updated successfully."
3 }
```



The screenshot shows a REST client interface with the following details:

- Status: 200 OK
- Size: 275 Bytes
- Time: 15 ms
- Response tab is selected, showing the following JSON array:

```
1 [
2   {
3     "id": 1,
4     "nome": "Nome 1",
5     "email": "nome1@dominio.com"
6   },
7   {
8     "id": 2,
9     "nome": "Ana Souza Silva",
10    "email": "nome2@dominio.com"
11  },
12  {
13    "id": 3,
14    "nome": "Nome 3",
15    "email": "nome3@dominio.com"
16  },
17  {
18    "id": 6,
19    "nome": "Nome 4",
20    "email": "nome4@dominio.com"
21  },
22  {
23    "id": 8,
24    "nome": "Nome 5",
25    "email": "nome5@dominio.com"
26  }
27 ]
```

Teste #6 - Atualização de usuário inexistente

- Método: PUT
- Endpoint: /usuarios/9999
- Payload: indiferente

Ao fazermos essa requisição, tentando atualizar um usuário inexistente, temos um código de status 404, já que o usuário não existe.

Ao enviarmos o seguinte payload:

```
{  
  "nome": "XXXXXXXXXXXXXXXXXXXXXXX",  
  "email": "YYYYYYYYYYYYYYYYYYYYYYY"  
}
```

Temos o seguinte resultado:

Status: 404 Not Found Size: 29 Bytes Time: 19 ms

Response Headers 7 Cookies Results Docs

```
1 {  
2   "message": "User not found."  
3 }
```

</> ≡

Copy

Teste #7 - Remover um usuário existente

- Método: DELETE
- Endpoint: /usuarios/8
- Payload: nenhum

Ao fazermos essa requisição, deletando um usuários existente de ID 8, temos código de status 200, já que tivemos sucesso em deletar um item:

```
Status: 200 OK   Size: 40 Bytes   Time: 51 ms

Response  Headers 7  Cookies  Results  Docs  </>  ≡
1  {
2    "message": "User deleted successfully."
3  }
```

Teste #8 - Remover um usuário inexistente

- Método: DELETE
- Endpoint: /usuarios/9999
- Payload: nenhum

Ao fazermos essa requisição, tentando deletar um usuários existente de ID 9999, temos código de status 404, já que esse usuário não existe:

```
Status: 404 Not Found   Size: 29 Bytes   Time: 15 ms

Response  Headers 7  Cookies  Results  Docs  </>  ≡
1  {
2    "message": "User not found."
3  }
```